

Bloque 02_metodologia

Índice de módulos

- Construcción y adaptación de payloads
- Metodología de laboratorio y room

Construcción y adaptación de payloads

Objetivos de aprendizaje

Construir una prueba desde el contexto, justificar cada componente, seleccionar evidencia in-band o blind y adaptar una restricción sin recurrir a listas ciegas.

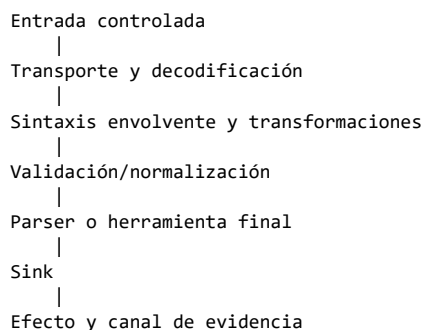
Prerrequisitos

HTTP, formatos de cuerpo, URL, encoding, parsers, flujo source-sink, shell, argumentos, SQL y permisos.

Por qué no memorizar cadenas

Un payload es una hipótesis serializada para un parser concreto. La misma cadena puede ser código en una shell, texto en argv, un valor JSON inválido o datos escapados en SQL. Memorizar oculta cinco preguntas: dónde se inserta, qué sintaxis existe alrededor, qué transformaciones ocurren, qué componente interpreta y qué evidencia puede observarse.

Modelo mental



Método de construcción

1. Copia una entrada válida.
2. Marca el punto exacto de inserción.
3. Escribe la sintaxis antes y después del valor.
4. Define una primitiva mínima: cambiar booleano, leer archivo conocido, ejecutar id, provocar una petición a un listener propio o alterar un argumento inocuo.
5. Decide si necesitas cerrar sintaxis y si debes reabrirlo.
6. Serializa para la capa exterior: URL, formulario, JSON, XML o multipart.
7. Predice respuesta y control negativo.
8. Ejecuta una sola variación.
9. Clasifica el fallo antes de adaptar.

Contextos de interpretación

Contexto	Unidad estructural	Control básico
SQL	token, literal, expresión, cláusula	consulta parametrizada
Shell	palabra, operador, expansión, redirección	ejecución sin shell y argv fijo
Argumentos	elemento de argv, opción, operando	--, allowlist y API segura
Ruta	segmento, raíz, enlace, ruta canónica	identificador indirecto y canonicalización
URL/SSRF	esquema, autoridad, host, puerto, path	parser único y política sobre destino efectivo
Plantilla	texto, expresión, sentencia	no evaluar plantillas controlables
Upload	metadatos, bytes, ubicación, servidor posterior	almacenamiento aislado y nombre generado
Autenticación	identidad, factor, estado, transición	servidor como autoridad de estado

Cierre, reapertura y separadores

Si el valor se inserta dentro de un literal SQL, la comilla puede cerrar ese literal; después se aporta una expresión y se neutraliza o reconstruye el sufijo. Si se inserta como número, una comilla puede ser innecesaria y romper la prueba. En shell, ;, &&, | y salto de línea solo son separadores si una shell los analiza. En argv pueden ser caracteres ordinarios.

Quoting pertenece a una capa. Las comillas que preservan JSON no son automáticamente las que necesita Bash o SQL. Dibuja de fuera a dentro:

```
PowerShell/curl -> JSON -> valor -> shell remota
```

Escapa cada capa una vez y verifica los bytes recibidos cuando el laboratorio lo permita.

Encoding y normalización

Codifica para transportar caracteres reservados; no lo uses como sinónimo de evasión. Para estudiar un filtro, registra:

- forma enviada;
- forma tras cada decodificación;
- comparación aplicada;
- forma que alcanza el sink;
- número de decodificaciones.

Un bypass solo está explicado cuando se identifica una discrepancia concreta entre validación y consumo.

Argumentos y opciones

Command injection introduce gramática de shell. Argument injection consigue elementos adicionales de argv. Option injection hace que un operando sea interpretado como opción. Prueba mínima: lograr una opción de salida inocua o un error inequívoco de la herramienta, no ejecutar código de inmediato. Consulta --help/manual de la versión detectada; -- no es universal.

Protocolos alternativos

En SSRF o clientes de URL, un esquema alternativo solo importa si la biblioteca lo soporta y el destino ofrece ese protocolo. Primero demuestra una petición HTTP a un listener propio; después verifica esquemas permitidos. No infieras soporte de `file:`, `gopher:` u otros por conocerlos de memoria.

Blacklist y allowlist

Una blacklist compara representaciones peligrosas conocidas y suele fallar frente a formas equivalentes. Una allowlist correcta expresa la propiedad permitida y se aplica tras parsing/canonicalización. Incluso una allowlist de host debe comprobar IP resuelta, puerto, redirecciones y cambios de resolución según el modelo de amenaza.

Canales de evidencia

- In-band: el resultado aparece en la respuesta.
- Booleano: dos condiciones producen respuestas distinguibles.
- Error: un error controlado revela parser o datos.
- Temporal: una demora depende de la condición y se compara con varias muestras.
- OOB/callback: el servidor contacta un endpoint HTTP o DNS propio del laboratorio.

Para pruebas blind usa controles negativos y repetición. Un retraso aislado o una resolución DNS no atribuible no confirma la primitiva completa.

Diagnóstico general

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
400 inmediato	transporte o formato inválido	enviar el original y cambiar un byte	corregir serialización
misma respuesta	entrada ignorada, caché o evidencia invisible	marcador único y control negativo	buscar otro canal
error de parser	contexto/sintaxis incorrectos	reducir a delimitador mínimo	ajustar cierre/reapertura
filtro explícito	comparación textual	comparar formas antes/después	corregir propiedad o estudiar normalización
efecto parcial	sink alcanzado con restricciones	primitiva más pequeña	adaptar a permisos/capacidad
comportamiento inestable	tiempo, estado o balanceo	repetir y fijar sesión	medir distribución

Reconocer la herramienta

Los mensajes de uso, opciones, versión, cabeceras y formato de error son huellas. Busca la frase exacta en documentación oficial, confirma versión y reproduce localmente con datos inocuos. No copies una opción de otra versión ni una sintaxis de un wrapper diferente.

Casos de construcción progresiva

SQL injection

Contexto: `SELECT * FROM items WHERE id = <entrada>`; entrada numérica. Primitiva: cambiar una condición. Construcción: conserva primero `id=7`; compara una expresión verdadera y una falsa válidas para el motor. Si ambas respuestas son iguales, no añadas extracción: comprueba caché, conversión a entero y si el valor llega a SQL. En contexto de texto, primero habría que cerrar el literal; en numérico no. Confirmación: diferencias reproducibles y controladas, no solo un error. Defensa: parámetros y allowlist para identificadores.

Command injection

Contexto interno hipotético: `sh -c "ping -c 1 <host>".` Primitiva: `id` en laboratorio. El payload interno `127.0.0.1; id` contiene operando válido, separador de shell y segundo comando. Al enviarlo como JSON, las comillas exteriores pertenecen a JSON, no a la shell. Si `;` se bloquea, pregunta si otra gramática ya aceptada permite secuencia; no pruebes separadores al azar. Si la app usa `execve(["ping", "-c", "1", host])`, cambia la hipótesis a argumentos. Defensa: evitar shell, validar host semánticamente y fijar argumentos.

Argument y option injection

Contexto: la app ejecuta `curl <url>` mediante lista de argumentos. `;` `id` no se interpreta. Primitiva: provocar una opción inocua reconocida por esa versión. Construye una entrada que el wrapper divida en más de un argumento solo si existe evidencia de splitting. Un error `unknown option` demuestra llegada al parser de opciones, no ejecución. Defensa: un valor por elemento, `--` si está soportado, `allowlist` de esquema/destino y API que no reinterprete texto.

Path traversal

Contexto: `open('/srv/images/' + filename).` Primitiva: leer un archivo conocido del laboratorio. Construye `../` por nivel desde la base conocida y normaliza mentalmente. Si se elimina una secuencia, compara original, filtrado y ruta canónica; una variante codificada solo tiene sentido si alguna capa decodifica después del filtro. Confirmación: contenido inequívoco o error que revele ruta resuelta, con control de archivo inexistente. Defensa: IDs indirectos, `allowlist` y comprobación de ruta canónica dentro de la base.

LFI

Contexto: `include(page + '.php')`. A diferencia de traversal de lectura, `include` puede interpretar contenido. Empieza con una vista válida, prueba ruta conocida y observa si el sufijo se añade. Los wrappers PHP solo aplican si PHP y esa configuración los soportan. El comando interno `curl -s '$URL/?page=../../../../etc/passwd'` transporta una ruta; las comillas protegen `$URL` en la shell local si son literales, por lo que en práctica debe adaptarse correctamente. Defensa: mapa fijo de identificador a plantilla.

SSRF

Contexto: el servidor obtiene la URL enviada. Primitiva: callback HTTP a un listener propio. El playbook usa `curl -sS '$URL/fetch?url=http://ATTACKER_IP:8000/':` primero sustituye variables y confirma la petición. Después compara loopback y destino inexistente solo en el laboratorio. Ante filtro, estudia parser, resolución y redirecciones; no asumas que otra escritura representa el mismo destino. Defensa: `allowlist` de destinos, resolución y validación de IP efectiva, bloqueo de redes no permitidas y revalidación tras redirección.

File upload

Capas: nombre declarado, Content-Type, bytes, nombre almacenado, ruta pública y handler del servidor. Primitiva: subir un archivo de texto marcador y recuperarlo. Luego verifica qué capa decide extensión/tipo. El ejemplo interno `curl -sS -F 'file=@shell.phtml;type=image/png' '$URL/upload'` mezcla bytes de un fichero con un tipo declarado; no demuestra por sí solo que se ejecute. Confirmación: recuperación, ubicación y, solo después, interpretación controlada. Defensa: nombre aleatorio, almacenamiento fuera del webroot, inspección de contenido y servidor sin ejecución.

SSTI

Contexto: entrada insertada en una plantilla evaluada. Primitiva: expresión aritmética inocua. `{{7*7}}` solo tiene significado para ciertas sintaxis; 49 reflejado confirma evaluación si el control literal no produce lo mismo. Después identifica motor y sandbox mediante mensajes/documentación antes de ampliar. Defensa: no compilar plantillas controladas y pasar datos como valores.

Autenticación

No siempre hay "cadena payload". Contexto: transición login -> MFA -> sesión autorizada. Primitiva: demostrar que el servidor acepta una transición no válida, por ejemplo manipulando un parámetro cuyo estado debería ser servidor. Cambia una variable, conserva sesión y compara acceso directo al recurso. Una pantalla omitida no confirma bypass si el backend rechaza la acción. Defensa: máquina de estados en servidor, rotación de sesión y autorización por petición.

Escalada de privilegios

El payload se construye desde una relación de permisos. Ejemplo sudo: salida de sudo -l -> regla concreta -> consulta de comportamiento documentado del binario -> prueba inocua -> capacidad efectiva. Ejemplo PATH: proceso privilegiado invoca nombre sin ruta -> directorio escribible precede al legítimo -> ejecutable marcador -> evidencia de identidad efectiva. No empieces con una reverse shell. Defensa: rutas absolutas, entorno controlado, privilegio mínimo y archivos no escribibles por identidades inferiores.

Anatomía obligatoria al documentar un payload

Contexto de entrada:
Sintaxis original:
Primitiva:
Payload interno:
Serialización de transporte:
Significado de cada componente:
Resultado esperado:
Control negativo:
Restricción observada:
Adaptación y motivo:
Evidencia final:

Errores frecuentes

- Escapar para la shell local y creer que se escapó para el parser remoto.
- Añadir varias técnicas a la vez y perder causalidad.
- Confundir reflexión con ejecución.
- Confundir error con confirmación.
- Copiar una variante de otro motor, versión o sistema operativo.
- ampliar impacto antes de demostrar la primitiva.
- usar callbacks fuera del entorno autorizado.

Resumen

Un payload correcto une contexto, gramática, transformación, sink y evidencia. La adaptación no consiste en cambiar caracteres hasta obtener respuesta, sino en probar una explicación sobre el procesamiento.

Chuleta operativa

1. Entrada y formato.
2. Sintaxis envolvente.
3. Parser final.
4. Primitiva mínima.
5. Serialización exterior.
6. Resultado y control negativo.
7. Una variación.
8. Clasificar fallo.
9. Adaptar con motivo.
10. Registrar evidencia y mitigación.

Referencias

- [Web Security Academy: materiales](#)
- [GNU Bash Reference Manual](#)
- [Conceptos internos](#)

- [Metadatos internos de comandos](#)

Anterior: [Metodología de laboratorio](#). Siguiendo: módulos de 03_seguridad_web/.

Metodología de laboratorio y room

Objetivos

Pasar de evidencia a hipótesis comprobable, diseñar pruebas mínimas, reconocer niveles de confirmación y abandonar líneas sin soporte.

Flujo general

1. Confirma alcance, objetivo, restricciones y horario.
2. Crea inventario de hosts, nombres, puertos, servicios, tecnologías e identidades.
3. Convierte cada observación en una hipótesis concreta.
4. Define la evidencia que confirmaría y la que refutaría.
5. Ejecuta la prueba mínima de menor riesgo.
6. Interpreta resultado y control negativo.
7. Amplía solo si cambia la conclusión o el impacto.
8. Registra evidencia y siguiente decisión.

Plantilla de hipótesis

Observación	Hipótesis	Prueba mínima	Resultado esperado	Resultado obtenido	Conclusión

Plantilla de entrada a sink

- Entrada controlada:
- Transformaciones conocidas:
- Componente que procesa la entrada:
- Sink potencial:
- Restricciones observadas:
- Primitiva que quiero demostrar:
- Evidencia esperada:
- Payload mínimo:
- Adaptaciones posibles:

Mapa de superficie

Organiza por frontera, no por herramienta:

Frontera	Activos	Entradas	Identidad	Dependencia	Evidencia
Internet-web	vhost/API	path, query, body, headers	anónima/usuario	DB, filesystem	petición-respuesta
Host local	procesos/servicios	config, entorno, archivos	usuario efectivo	kernel, sudo	permisos y ejecución
Dominio	cuentas/equipos	LDAP, SMB, Kerberos	principal	DC, AD CS	relaciones/tickets

Niveles de certeza

- No confirmada: posibilidad basada en diseño o tecnología.
- Indicio: comportamiento compatible, pero con explicaciones alternativas.
- Confirmación: prueba reproducible demuestra la primitiva.
- Explotación: se usa la primitiva para obtener una capacidad adicional.
- Impacto: consecuencia sobre confidencialidad, integridad o disponibilidad dentro del alcance.

No llares SQLi a un 500 ni SSRF a una demora aislada. Añade controles: entrada inocua, destino inexistente, repetición y comparación.

Cuando falla la prueba básica

Clasifica antes de adaptar:

1. transporte incorrecto;
2. entrada no controlada;
3. parser distinto;
4. sintaxis/contexto incorrectos;
5. normalización o filtro;
6. sink no alcanzado;
7. efecto sin canal observable;
8. hipótesis falsa.

Cada adaptación debe discriminar entre dos causas. Probar listas aleatorias aumenta ruido sin aumentar conocimiento.

Investigar herramientas y errores

Extrae nombre, versión, sintaxis mostrada y fase del error. Consulta primero manual oficial, ayuda local y documentación del proyecto. Reproduce la mínima invocación en un entorno propio. Un mensaje puede proceder de proxy, framework, librería, herramienta externa o sistema operativo; atribuirlo al componente equivocado produce payloads inútiles.

Abandonar una hipótesis

Detente cuando la entrada no llega al componente, controles positivos y negativos son indistinguibles, la evidencia contradice el mecanismo, o el coste/riesgo supera el valor. Registra qué se descartó y bajo qué condiciones; podría reabrirse con nueva evidencia.

Pistas progresivas

1. identificar la entrada;
2. nombrar el componente;
3. indicar la primitiva;
4. señalar la restricción;
5. mostrar un fragmento, nunca la solución completa antes del intento.

Referencias

- [OWASP Web Security Testing Guide](#)
- [Guías internas](#)

Anterior: [Flujo de datos](#). Siguiendo: [Construcción de payloads](#).